

3-WEEK

SPRINT PLAN

Complete Daily Task Breakdown

All Features · All Members · Every Day · February → May 2025

AI · OCR · RAG · LLM · Agents · PDF · Frontend · Docker · Chatbot

WEEK 1

Foundation & Core AI
5 DAYS

WEEK 2

PDF & Chatbot
5 DAYS

WEEK 3

Demo & Launch
5 DAYS

TEAM MEMBERS

YOUSSEF B.

Team Leader · AI Engineer

YOUSTINA

OCR · Data · Pipeline

HAGER

RAG · Legal KB

YOUSSEF R.

Agents · PDF · Frontend

15

DAYS

4

ENGINEERS

8

SERVICES

3

FEATURES

45+

LAW ARTICLES

20

TEST CASES

WEEK 1FOUNDATION & CORE AI

TRL 1 → TRL 3

Setup · Data · OCR · Classification · RAG

5 Days · 20 Tasks · All Members

WEEK GOAL:

End of Week 1: GitHub repo live · Docker running on all 4 machines · Legal KB indexed · OCR working · LLM classifier returns structured JSON · 20 test cases ready

DAILY OVERVIEW

MON Day 1 Project Kickoff & Environment BACKEND DOCKER FRONTEND	TUE Day 2 OCR Service + Data Ingestion BACKEND FRONTEND OCR	WED Day 3 Google Vision Fallback + Entity Extraction BACKEND FRONTEND OCR	THU Day 4 LLM Classification + RAG Indexing BACKEND LLM FRONTEND	FRI Day 5 Verification Agents + Gate Review TEST LLM AGENTS
--	--	--	---	--

WEEK GATE — Must pass ALL before next week:

- ✓ All 20 test cases return valid JSON (0 schema errors)

✓ OCR reads Arabic from screenshots (>70% accuracy)

✓ LLM classifies claims without evidence_block_id citation

✓ ChromaDB returns ≥1 article for each of 4 crime types

✓ CI pipeline green on GitHub

DAY 1 MON

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

Project Kickoff & Environment

5 tasks across all team members

Youssef B DOCKER Create GitHub repo + branch strategy

- Create org/team repo on GitHub
- Create branches: main, develop, feature/m1-*, feature/m2-*, feature/m3-*, feature/m4-*
- Set branch protection rules (main = 2 approvals, develop = 1)
- Add .gitignore, README skeleton, .env.example
- Share repo access to all 4 members

Youssef B BACKEND Design unified JSON schema

- Draft evidence_block schema: {block_id, file_name, raw_text, confidence, entities}
- Draft case schema: {case_id, crime_type, claims, articles, score, timeline}
- Share schema doc in /docs/schema.md
- All 4 members review and agree — this is the API contract

Youstina DOCKER Docker environment setup

- Create docker-compose.yml with all 8 services (ports 8000-8006, 8501)
- Create Dockerfile for each service (FROM python:3.11-slim)
- Add healthcheck for each service
- Test: docker-compose up starts all services
- Document in README: how to run locally

HS DATA Download & parse all law sources

- Download Law 175/2018 Arabic from wipo.int/wipolex/ar/legislation/details/19959
- Download Egyptian Penal Code from hrlibrary.umn.edu
- Parse PDF → per-article JSON: {article_id, law, text, penalty, crime_type}
- Confirm all 45 articles of Law 175/2018 covered
- Store in data/law/ directory

Youssef R FRONTEND Streamlit UI skeleton

- Create frontend/ directory with Streamlit app.py
- Build Upload page: drag-drop or select files
- Build Results page: placeholder for score + summary
- Build Chat page: placeholder for chatbot
- Confirm it runs: streamlit run app.py

DAY 2TUE

TEAM: YB YoussefYS YoustinaHS HagerYR Youssef

OCR Service + Data Ingestion

5 tasks across all team members

YoustinaOCRBuild OCR service (EasyOCR)

- Install EasyOCR in services/ocr/
- Build POST /ocr endpoint (FastAPI)
- Input: image file (bytes) → Output: list of evidence_blocks JSON
- Arabic+English detection: reader = easyocr.Reader(["ar", "en"])
- Return avg_confidence per block
- Test on 5 WhatsApp screenshots (Arabic)

< GUIDE AVAILABLE → GUIDE: pip install easyocr. reader = easyocr.Reader(["ar","en"], gpu=False). results = reader.readtext(image_bytes, deta..

YoustinaOCRArabic text normalisation

- Strip diacritics (remove \u064B-\u065F)
- Unify Alef variants (إِإ → I)
- Fix Ta marbuta (ة → ة for matching)
- Use arabic-reshaper + python-bidi for correct display
- Write normalize_arabic(text: str) → str function
- Add unit test: 5 normalisation cases

HSHagerDATAGenerate 20 synthetic test cases

- Use Claude/Gemini to generate 5 cases each: blackmail, scam, threat, defamation
- Each case: 2-3 screenshot text samples + expected crime_type + expected entities
- Store as JSON in data/test_cases/ — no real victim data
- Label each with ground truth: crime_type, extracted phones/amounts/dates
- Review all 20 for quality and realism

YB Youssef BBACKENDMain API orchestrator skeleton

- Create services/api/ with FastAPI app
- Build POST /analyze endpoint (accepts files, returns case JSON)
- Wire sequential calls: OCR → classify → retrieve → verify → score → pdf
- Add case_id generation (UUID4)
- Add error handling: if any service fails, return partial result with error flag

YR Youssef RFRONTENDConnect upload to OCR

- Wire Streamlit upload to POST /ocr endpoint
- Display extracted text blocks in UI
- Show confidence score per block (colour: green >0.75, yellow >0.5, red <0.5)
- Show extracted entities (phone numbers, amounts) highlighted
- Add loading spinner during processing

DAY 3 WED

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

Google Vision Fallback + Entity Extraction

5 tasks across all team members

YS Youstina

OCR

Google Vision API fallback

- Add Google Vision as fallback when EasyOCR confidence < 0.65
- pip install google-cloud-vision
- Store GOOGLE_API_KEY in .env (never hardcode)
- Function: call_google_vision(image_bytes) → evidence_blocks
- Test: use a blurry/dark screenshot that fails EasyOCR
- Log which source was used: easyocr or google_vision

< GUIDE AVAILABLE → GUIDE: from google.cloud import vision. client = vision.ImageAnnotatorClient(). image = vision.Image(content=image_bytes..

HS Hager

DATA

Entity extraction with regex patterns

- Write Egyptian phone regex: r"(\+20|0)(10|11|12|15)\d{8}"
- Write amount regex: r"\d{1,7}[.]?[0-9]*\s*(ج.م|ج.د|ل.م|ل.د|EGP|LE)"
- Write date regex: r"\d{1,2}/[-]\d{1,2}/[-]\d{2,4}"
- Write URL regex: r"https?://[S+|www\.\S+"
- Extract IBAN/account numbers for fraud cases
- Return entities dict: {phones:[], amounts:[], dates:[], urls:[]}

HS Hager

DATA

AraBERT NER for names/orgs

- pip install camel-tools transformers
- Use CAMEL Tools for Egyptian Arabic NER
- Extract person names and organisation names from evidence text
- Add to entities dict: {persons:[], organisations:[]}
- Test on 5 synthetic cases — confirm extraction works

< GUIDE AVAILABLE → GUIDE: from camel_tools.ner import NERrecognizer. ner = NERrecognizer.pretrained(). sentences = text.split("."). entities ..

YB Youssef B

BACKEND

Schema validation with Pydantic

- Create models.py: EvidenceBlock, CaseOutput, ClaimItem, ArticleRef Pydantic models
- All service outputs must validate against these models
- Add validation at API layer: reject malformed responses
- Write test: 10 valid + 3 invalid schema cases
- This ensures pipeline data integrity

YR Youssef R

FRONTEND

Entity display in UI

- Show extracted entities in UI cards
- Phone numbers: clickable tel: links
- Amounts: highlighted in gold
- Dates: sorted timeline preview
- Add "Entity not found" message if extraction returns empty

DAY 4 THU

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

LLM Classification + RAG Indexing

5 tasks across all team members

Youssef B LLM Crime classification prompt (citation-forcing)

- Write system prompt that FORCES citation of evidence_block_id for every claim
- Structure: "You may ONLY make a claim if you cite the evidence_block_id that supports it"
- Output schema: {crime_type, confidence, claims: [{claim, evidence_block_ids}], missing_evidence: []}
- Test with Claude Sonnet 4.6 and Gemini 1.5 Flash (free tier)
- Validate: no claim should appear without a block_id

✂️ GUIDE AVAILABLE → GUIDE: Citation-forcing is the KEY design. In the prompt: "ABSOLUTE RULE: For every claim you make, you MUST cite at least one evidence_block_id"

HS Hager RAG Index Egyptian law into ChromaDB

- pip install chromadb langchain-chroma sentence-transformers
- Load parsed law JSON from data/law/
- One chunk = one article (never split articles)
- Embed with multilingual-e5-large: HuggingFaceEmbeddings(model_name="intfloat/multilingual-e5-large")
- Store metadata: {article_id, law, crime_type, penalty}
- Persist: vectorstore.persist() to data/law_db/

✂️ GUIDE AVAILABLE → GUIDE: from langchain_community.vectorstores import Chroma. from langchain_community.embeddings import HuggingFaceEmbeddings

HS Hager RAG RAG retrieval service

- Build POST /retrieve endpoint in services/rag/
- Input: {case_text, crime_type} → Output: list of matching articles
- Use crime_type as hard filter: filter={"crime_type": crime_type}
- Retrieve top 5 articles by cosine similarity
- Add citation validator: every article must exist in ChromaDB before returning
- Test: blackmail query → returns Art.25, Art.26 (Law 175/2018)

YR Youssef R FRONTEND Crime type display + confidence meter

- Display classified crime type with icon
- Show confidence as animated progress bar
- Color: green ≥0.8, yellow ≥0.6, red <0.6
- Show retrieved law article summaries as expandable cards
- Add "Disputed" badge if confidence <0.6

YB Youssef B BACKEND Wire OCR → Classify → RAG pipeline

- Connect OCR output to classifier input in orchestrator
- Pass classifier output to RAG retriever
- Handle Pydantic validation at each step
- Add timing logs: log time for each stage
- Test full chain on 3 synthetic test cases

DAY 5 FRI

Verification Agents + Gate Review

5 tasks across all team members

TEAM: YB YS YS HS YR

Youssef Youstina Hager Youssef

YR AGENTS

Attacker agent (LangGraph)

- pip install langgraph
- Build Attacker agent: role = hostile defense lawyer
- For each claim, verdict: SUPPORTED or UNSUPPORTED
- Verdict criteria: (1) block_id cited, (2) block actually contains claim text, (3) not inference without evidence
- Return: {attacks: [{claim, verdict, reason}]}
- Test: plant 3 weak/unsupported claims → Attacker must find all 3

⚡ **GUIDE AVAILABLE** → GUIDE: from langgraph.graph import StateGraph. Define VerificationState(TypedDict): claims, evidence_blocks, attacks, ve...

YR AGENTS

Judge agent + revision loop

- Build Judge agent: reviews Attacker findings
- Verdicts: APPROVED (0 unsupported) or NEEDS_REVISION (n unsupported)
- Auto-remove unsupported claims from report when NEEDS_REVISION
- Loop: max 3 rounds, then return NEEDS_USER_REVIEW if still failing
- Log all rounds: {round, attacks_found, verdict} for audit trail

HS RAG

Citation validator function

- Write validate_citations(articles, crime_type) function
- For each article: verify article_id exists in ChromaDB with matching crime_type
- Return: {valid: [], invalid: [], status: PASSED|FAILED}
- Any invalid → remove from output, log warning
- Add to POST /retrieve response: citation_validation_status

⚡ **GUIDE AVAILABLE** → GUIDE: result = vectorstore.get(where={"article_id": article["article_id"]}). If result["documents"] is empty → article ..

YB LLM

Evidence Strength Score engine

- Implement 0-100% scoring with weights
- Weights: explicit_threat=20, financial_demand=20, contact_identified=15, multiple_files=15, ocr_confidence=15, evidence_strength=15, media_type=5
- grades: STRONG ≥ 75, MEDIUM ≥ 48, WEAK < 45
- Return breakdown: {total_score, grade, breakdown:{...}}
- Test: high-quality case should score STRONG

YS TEST

Week 1 Gate Tests

- Run all 20 synthetic test cases through OCR → classify → RAG pipeline
- Confirm: all 20 return valid JSON (no schema errors)
- Confirm: 0 claims without evidence_block_id
- Confirm: ChromaDB returns at least 1 article for each of 4 crime types
- Write summary report: what passed, what failed, what to fix in Week 2
- Gate check: ALL of the above must pass before Week 2 starts

WEEK 2PDF GENERATION & CHATBOT

TRL 3 → TRL 4

Verification · Arabic PDF · Legal Chatbot · Integration

5 Days · 20 Tasks · All Members

WEEK GOAL:
End of Week 2: Full Arabic RTL complaint PDF generating correctly · Legal chatbot answering in Arabic with law citations · All 8 services running in Docker · E2E pipeline tested

DAILY OVERVIEW

MON5 Day 6 Arabic PDF Template + WeasyPrint Setup BACKEND PDF TEST	TUE5 Day 7 PDF Generation Service + Score Visualisation LLM DOCKER PDF	WED5 Day 8 Legal Chatbot Build BACKEND LLM INFRA	THU5 Day 9 Full E2E Integration + GitHub Actions CI DOCKER INFRA BACKEND	FRI5 Day 10 Polish, Fixes & Week 2 Gate Review TEST LLM PDF
--	--	--	--	---

WEEK GATE — Must pass ALL before next week:

✓ Arabic PDF renders correctly on Windows + Android

✓ 20/20 test cases pass E2E pytest suite

✓ E2E pipeline: upload → PDF in ≤90 seconds

✓ Chatbot answers in Arabic, cites correct law articles

✓ 0 API keys committed to Git (secrets audit)

DAY 6MON

TEAM: YB YoussefYS YoustinaHS HagerYR Youssef

Arabic PDF Template + WeasyPrint Setup

5 tasks across all team members

Youssef RPDFWeasyPrint + Amiri font setup

- pip install weasyprint jinja2
- Download Amiri font: place in assets/fonts/Amiri-Regular.ttf
- Test: generate a simple Arabic RTL paragraph PDF
- Confirm right-to-left text renders correctly
- Confirm Arabic ligatures (letter joining) work
- If rendering issues: add @font-face in CSS + dir=rtl on <html>

< GUIDE AVAILABLE → GUIDE: from weasyprint import HTML. Install Pango + libcairo: apt-get install libpango-1.0-0 libpangocairo-1.0-0 libcair..

Youssef RPDFJinja2 complaint HTML template

- Create services/pdf_gen/templates/complaint_ar.html
- Section 1: Cover (case_id, crime type, date, victim info)
- Section 2: Case summary (AI narrative in formal Arabic)
- Section 3: Evidence timeline (dates chronological)
- Section 4: Law articles (full text + penalty)
- Section 5: Evidence Strength Score badge (green/amber/red)
- Section 6: Verification badge + next steps (108 hotline)

HSRAGFull law article text for PDF

- Ensure each ChromaDB article stores full Arabic text (not just summary)
- Add penalty_ar field: full Arabic penalty text
- Add article_number field separate from article_id
- Test: retrieve Art.26 → returns full Arabic text + penalty
- These will appear verbatim in the complaint PDF — must be 100% accurate

Youssef BBACKENDCase data assembly for PDF

- Write assemble_pdf_data(case_json) function
- Merges OCR, classification, RAG, verification, score into one PDF context dict
- Sorts evidence_blocks chronologically by date
- Formats all phone numbers, amounts for display
- Returns complete context dict for Jinja2 template

YSYoustinaTESTOCR accuracy benchmark

- Test OCR on all 20 test cases
- Calculate: word error rate vs ground truth text
- Target: >70% average word accuracy
- Identify failure cases (blurry, dark, unusual fonts)
- Document improvement actions needed (preprocessing, fallback tuning)

DAY 7 TUE

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

PDF Generation Service + Score Visualisation

5 tasks across all team members

YR Youssef R PDF Build POST /pdf endpoint

- Create services/pdf_gen/ FastAPI app
- POST /pdf: accepts full case_json → returns PDF bytes
- Call jinja2.render(template, context) → html_string
- Call weasyprint.HTML(string=html_string).write_pdf() → bytes
- Return as FileResponse with Content-Type: application/pdf
- Test: curl -X POST /pdf -d @case.json --output test.pdf

YR Youssef R PDF Score badge + verification stamp

- CSS: .score-badge { background: green if ≥75, amber if ≥45, red if <45 }
- Add circular score meter (CSS border-radius animation)
- Add "AI VERIFIED" stamp: ✓ Attacker Agent: 0 unsupported claims
- Add "تنترنال احابم" contact section: 108 · 0224065052 · moi.gov.eg
- Test PDF rendering on: Windows, macOS, Android (3 devices minimum)

YB Youssef B LLM Timeline constructor

- Write build_timeline(evidence_blocks, entities) function
- Extract all date references from evidence
- Sort chronologically
- Generate event summary per date in formal Arabic
- Return: [{date, event_summary, block_id}] sorted ascending
- Handle missing dates: place at end with "date unknown" label

HS Hager RAG Retrieval accuracy evaluation

- For each of the 20 test cases, check: correct articles retrieved?
- Metric: Precision@5 — of top 5, how many are relevant?
- Target: >80% precision across 20 cases
- Document which crime type has lowest precision
- Tune: adjust chunk size, metadata filters, or embedding query if needed

YS Youstina DOCKER Docker networking + service healthchecks

- Ensure all 8 services can reach each other via Docker network
- docker-compose.yml: each service has healthcheck: test: ["CMD", "curl", "-f", "http://localhost:PORT/health"]
- API orchestrator only calls services after healthcheck passes
- Test: kill one service → orchestrator should return error, not crash
- Add retry logic: 3 retries with 2s delay before failing

DAY 8 WED
Legal Chatbot Build

5 tasks across all team members

TEAM: Youssef Yousstina Hager Youssef

Youssef B LLM Legal chatbot system prompt

- Build chatbot prompt with full case context injection
- Rules: (1) Answer ONLY in Arabic, (2) cite article numbers for every legal statement, (3) if article not in retrieved list → say requires further consultation (4) never invent articles
- Inject retrieved articles in system prompt as reference
- Handle Egyptian Arabic dialect input → formal Arabic response
- Test: ask "إني زت ب ي ل ل ص خ ش ل ل ل ص ح ي ه ه ا" → must cite Art.25/26

GUIDE AVAILABLE → GUIDE: system_prompt = f""You are a certified Egyptian legal advisor. Case context: {case_context}. RULES: 1. Answer ON..

Youssef B BACKEND Chatbot session memory service

- Build services/chatbot/ FastAPI app
- POST /chat: {session_id, user_message, case_context} → {assistant_reply}
- Store conversation history in session_store dict (or Redis for production)
- Inject full history on every API call (Claude has no memory between calls)
- Limit history to last 20 turns to avoid token overflow
- POST /chat/reset: clear session history

Youssef R FRONTEND Chatbot UI in Streamlit

- Build chat interface: message bubbles (user right, AI left)
- User messages: dark card, cyan border
- AI messages: show law article citations as highlighted badges
- Add "Generate PDF from this conversation" button
- Add typing indicator (animated dots)
- Persist session across page navigation in Streamlit

HS TEST Chatbot accuracy testing

- Create 20 standard Q&A test questions (5 per crime type)
- Expected answers: what article should be cited?
- Test each question: does chatbot cite correct article?
- Target: ≥80% cite correct article, 0 invented articles
- Document failures for Youssef B to tune prompt

YS INFRA Environment variables + secrets management

- Audit all code: no hardcoded API keys anywhere
- All keys in .env file (gitignored)
- Add .env.example with placeholder values
- Add pre-commit hook: scan for common key patterns in staged files
- Document in README: how to set up .env for each team member

DAY 9THUFull E2E Integration + GitHub Actions CI5 tasks across all team members

TEAM: YB YoussefYS YoustinaHS HagerYR Youssef

Youssef BDOCKERFull E2E pipeline integration test

- Write tests/test_e2e.py using pytest
- Test: upload 3 test screenshots → trigger /analyze → receive PDF
- Assert: PDF bytes returned, file valid, >0 bytes
- Assert: case_json has crime_type, ≥1 article, score 0-100
- Assert: all Pydantic schemas pass validation
- Run: pytest tests/test_e2e.py -v on all 4 machines

< GUIDE AVAILABLE → GUIDE: import httpx. @pytest.mark.asyncio. async def test_analyze(): async with httpx.AsyncClient() as client: files = [...]

HS HagerINFRAGitHub Actions CI workflow

- Create .github/workflows/ci.yml
- Trigger: on push to develop and main
- Jobs: (1) pytest unit tests, (2) docker-compose build, (3) docker-compose up + health checks
- All CI jobs must pass before any PR can merge to develop
- Add status badge to README
- Test: push to develop → CI runs automatically

< GUIDE AVAILABLE → GUIDE: name: CI. on: push: branches: [main, develop]. jobs: test: runs-on: ubuntu-latest. steps: - uses: actions/checkou...

Youssef RFRONTENDFull UI flow integration

- Wire all UI pages together: Upload → Processing → Results → PDF → Chat
- Add progress stepper: showing which pipeline stage is running
- Show error states for each stage (OCR failed, classification failed, etc.)
- Add download button for PDF
- Make chat context-aware: automatically loads current case when opened

YS YoustinaTESTAll 20 test cases E2E run

- Run all 20 test cases through full pipeline
- Record: pass/fail per case, time taken, score assigned
- Target: ≥18/20 pass (90%)
- Target: ≤90 seconds average per case
- Document 2 failure cases for investigation on Day 10

Youssef BBACKENDAPI timing + error logging

- Add request timing middleware to main API
- Log: case_id, timestamp, each_stage_duration, total_duration
- Log errors with full traceback to logs/ directory
- Add GET /status/{case_id} endpoint for async processing status
- Add rate limiting: 10 requests/minute per IP (for production safety)

DAY 10FRI

TEAM: YB YoussefYS YoustinaHS HagerYR Youssef

Polish, Fixes & Week 2 Gate Review

5 tasks across all team members

ALLTESTBug fixes from Day 9 test run

- Fix all E2E failures found on Day 9
- Re-run pytest after each fix — do not batch fixes
- All 4 members: pair programming on hardest bugs
- Document: what broke, why, how fixed
- Target: 20/20 test cases passing before end of Day 10

YR Youssef RPDFPDF rendering QA across devices

- Open generated PDF on: Windows (Adobe Reader), macOS (Preview), Android (Drive), iOS (Files)
- Check: Arabic text direction correct on all
- Check: Amiri font renders, ligatures correct
- Check: no text overflow or missing sections
- Report any device-specific rendering issues

HS HagerRAGRAGLegal accuracy spot check

- Manually review 5 generated PDFs
- For each: are cited articles correct for the crime type?
- Are article texts quoted verbatim from source law?
- Are penalties quoted accurately?
- Note any inaccuracies for prompt tuning
- Keep log: accuracy_review.md in docs/

YB Youssef BLLMLLMPrompt optimization pass

- Review classifier prompt: are all 20 test cases classifying correctly?
- Review chatbot prompt: are article citations accurate?
- Tune temperature (lower = more consistent legal answers)
- Test: re-run any failed test cases after prompt changes
- Document all prompt versions in services/*/prompts/*.md

YS YoustinaTESTTESTWeek 2 Gate Review

- Gate check 1: Full pipeline runs in ≤90s on all 4 machines
- Gate check 2: Arabic PDF renders correctly on 3+ devices
- Gate check 3: Chatbot answers in Arabic, cites correct articles
- Gate check 4: CI pipeline green on GitHub
- Gate check 5: 0 API keys committed to Git
- Write Week 2 Summary: what achieved, what remains for Week 3

WEEK 3

DEMO, QUALITY & LAUNCH

TRL 4 → TRL 6

Cloud Deploy · Legal Review · Demo Video · Final Polish

5 Days · 20 Tasks · All Members

WEEK GOAL:

End of Week 3: Cloud demo deployed with live URL · Legal advisor validates 5 PDFs · Demo video recorded · Full documentation complete · Proposal submitted · Ready for presentation

DAILY OVERVIEW

MON Day 11 Cloud Deployment + Public Demo DOCKER INFRA TEST	TUE Day 12 Legal Advisor Review + Quality Hardening TEST LLM PDF	WED Day 13 Demo Video + Final Documentation DEMO INFRA TEST	THU Day 14 Presentation Prep + Final Checks DEMO PDF INFRA	FRI Day 15 LAUNCH DAY — Final Presentation DEMO ? HS VB
--	---	--	---	---

WEEK GATE — Must pass ALL before next week:

- ✓ Cloud demo deployed with public URL accessible

✓ Legal advisor rates $\geq 5/5$ PDFs as legally accurate

✓ Demo video (5 min) recorded and uploaded

✓ README scored $\geq 9/10$ on completeness checklist

✓ All 6 KPIs measured and documented with actual numbers

DAY 11 MON

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

Cloud Deployment + Public Demo

5 tasks across all team members

Youssef B INFRA Deploy to Railway or Fly.io

- Choose: Railway (easiest) or Fly.io (more control)
- Create railway.toml or fly.toml for each service
- Deploy all 8 services to cloud
- Configure environment variables in cloud dashboard (not .env)
- Test: curl https://your-app.railway.app/health on all services
- Set up custom subdomain if available

< GUIDE AVAILABLE → GUIDE – Railway: npm install -g @railway/cli. railway login. railway init. railway up. In dashboard: add all environment..

Youssef R FRONTEND Production UI polish

- Replace placeholder UI elements with final versions
- Add ACEB logo / shield icon to header
- Mobile responsive layout (works on phone browser)
- Add Arabic/English toggle button
- Performance: lazy load chat history
- Test on mobile browser (Chrome Android, Safari iOS)

HS DATA Prepare 3 demo cases

- Create 3 realistic anonymised demo cases (fictional characters)
- Case A: blackmail with photos — should score STRONG (>75%)
- Case B: financial scam — should score MEDIUM (50-75%)
- Case C: threat via WhatsApp — should score WEAK (<45%)
- Prepare screenshots in both Arabic and mixed Arabic/English
- These will be used in the demo video — practice walkthrough

YS Youstina TEST Load testing + performance

- Run 5 concurrent requests to /analyze — does it handle without crash?
- Measure: average response time under load
- Identify bottlenecks: which service is slowest?
- Consider: if OCR service is slow, add caching for same-file hash
- Document performance benchmark: time per stage in results

YB DOCKER Production docker-compose overrides

- Create docker-compose.prod.yml (production settings)
- Production: remove bind mounts, set restart: always, limit memory per service
- Add nginx reverse proxy for clean URLs
- Add SSL if deploying with custom domain
- Test: docker-compose -f docker-compose.yml -f docker-compose.prod.yml up

DAY 12 TUE

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

Legal Advisor Review + Quality Hardening

5 tasks across all team members

ALL TEST Legal advisor review session

- Present 5 generated complaint PDFs to a qualified Egyptian lawyer
- Review checklist: (1) Articles cited correctly? (2) Penalties quoted accurately? (3) Language formal and appropriate? (4) Structure matches standard complaint format? (5) Contact info correct?
- Document all feedback into docs/legal_review.md
- Implement all critical legal accuracy fixes same day
- Target: ≥90% accuracy rating from advisor

HS RAG Fix any legal inaccuracies found

- Update ChromaDB with corrected article texts if any errors found
- Re-validate all test cases after changes
- Add note in code: "Article X updated per legal review [date]"
- Consider: add "last verified" date to each article in metadata
- Re-run citation validator on all 20 test cases after any changes

YR PDF PDF formal language improvements

- If legal advisor suggests language changes: update Jinja2 template
- Replace informal Arabic with formal legal Arabic phrases
- Add standard complaint header format if advised
- Update "Next Steps" section with any new information from advisor
- Re-generate all 20 test PDFs with updated template

YB LLM Chatbot legal accuracy hardening

- After legal review: update chatbot system prompt with any corrections
- Add explicit prohibition: never quote penalties not in retrieved articles
- Add caveat in chatbot UI: "This is AI-assisted guidance, not legal advice"
- Test: the 20 chatbot Q&A cases — are all citations correct post-fix?
- Add fallback: if question outside scope → recommend تدرت إلّا تحابم hotline 108

YS TEST Security audit basics

- Check: no API keys in git log (git log --all | grep -i "key|secret|password")
- Check: input validation on /analyze (max file size 10MB, allowed types PNG/JPG/PDF only)
- Check: check file type by magic bytes, not just extension
- Add rate limiting headers to responses
- Test: upload a .exe renamed as .png → must be rejected

DAY 13 WED

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

Demo Video + Final Documentation

5 tasks across all team members

ALL DEMO Record 5-minute demo video

- Script: (1) Problem statement 30s, (2) Upload evidence 45s, (3) Analysis + score 30s, (4) Download PDF + show sections 60s, (5) Playback of A-60s, (6) Closing Try reader 15s
- Record with screen recording + voiceover (Arabic+English)
- Show all 3 demo cases (A=STRONG, B=MEDIUM, C=WEAK)
- Upload to YouTube/Drive as backup
- This video is the backup if live demo fails — must be perfect

YR Youssef R FRONTEND Final UI refinements

- Fix any UI bugs found during demo video recording
- Add loading animations that show pipeline stages progressing
- Ensure Arabic text renders correctly in browser (not just PDF)
- Add error boundary: friendly error page instead of crash
- Final accessibility check: readable font sizes, adequate contrast

HS Hager DATA Complete README.md

- Sections: Project Overview, Features, Architecture diagram (ASCII), Quick Start, Environment Variables, API Reference, Team, License
- Quick Start must work: git clone → docker-compose up → works
- Add screenshots of the UI to README
- Add link to demo video
- Self-rate against 10-point checklist: target ≥9/10

YB Youssef B INFRA Code documentation pass

- Every public function must have a docstring
- Format: one-line summary, Args:, Returns:, Raises: if applicable
- Run: pydoc check on all 8 services
- Fix: any function with missing docstring
- Target: 100% docstring coverage on public functions

YS Youstina TEST Final 20-case regression test

- After all changes: re-run all 20 test cases
- Confirm: still 20/20 passing (or document any new failures)
- Run pytest with --cov flag: confirm ≥75% test coverage
- Fix any coverage gaps in core pipeline functions
- Generate coverage report: pytest --cov=services/ --cov-report=html

DAY 14 THU

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

Presentation Prep + Final Checks

5 tasks across all team members

ALL DEMO Rehearse live demo (3x)

- Full live demo walkthrough: 3 complete runs on cloud URL
- Designate: presenter, backup person, chat helper
- Practice: what to do if cloud is slow (switch to local docker)
- Confirm: demo video is uploaded and accessible as backup
- Each member explains their own component (2 min each)

YB Youssef B INFRA Final submission checklist

- GitHub: main branch has final code, all PRs merged
- README: complete, accurate, demo link working
- Proposal PDF: this document — submitted
- Demo video: uploaded, link in README
- Docker: fresh git clone + docker-compose up tested by someone who has NOT seen the code
- All 6 KPIs: document current measured value vs target in docs/kpi_results.md

YR Youssef R PDF PDF final QA pass

- Final PDFs from 3 demo cases: print-ready quality check
- All Arabic text: correct direction, fonts, no garbled characters
- All law articles: match source documents verbatim
- Score badge: correct color for score
- Contact info: 108 hotline, 0224065052, moi.gov.eg all correct
- PDF metadata: title, author, subject set correctly

HS Hager DATA KPI measurement documentation

- KPI 1 — Data Quality: document final OCR accuracy, entity recall numbers
- KPI 2 — Model Performance: document classification accuracy on 20 cases
- KPI 3 — Pipeline Integration: document E2E success rate and timing
- KPI 4 — MLOps: confirm CI green, prompts versioned, no secrets in git
- KPI 5 — Output Quality: document legal advisor rating + user test results
- KPI 6 — Documentation: self-rate README, docstrings, proposal

YS Youstina TEST Zero-day bug hunting

- Spend 2 hours trying to break the system
- Try: very large files, corrupted images, empty files, non-Arabic text
- Try: extremely fast submissions (race conditions)
- Try: injection in text fields
- Document findings: what was found, what was fixed, what is known limitation

DAY 15 FRI
LAUNCH DAY — Final Presentation
5 tasks across all team members

TEAM: YB Youssef YS Youstina HS Hager YR Youssef

ALL DEMO MORNING: Final system health check

- 7:00 AM: run make health on cloud deployment — all 8 services green?
- Test Case A (STRONG blackmail) → confirm PDF downloads correctly
- Test chatbot: ask 3 questions → confirm Arabic answers with citations
- Confirm demo video link works
- Confirm proposal PDF submitted
- If any service down: switch to backup local docker immediately

ALL DEMO PRESENTATION: Project demo

- Slide 1: Problem — 70% never report, 0 Arabic tools
- Slide 2: Solution — ACEB 5-stage pipeline overview
- Slide 3: Live Demo — Case A (strong blackmail evidence)
- Slide 4: PDF Output — walk through all sections
- Slide 5: Chatbot Demo — live Q&A in Arabic
- Slide 6: Architecture — 8 microservices, Docker, CI/CD
- Slide 7: KPI Results — actual numbers vs targets
- Slide 8: TRL Roadmap — where we are, where we go

YB Youssef B DEMO Present: Architecture + LLM design

- Explain: why citation-forcing prompts prevent hallucination
- Explain: the Attacker+Judge agent loop and why it matters for legal accuracy
- Explain: ChromaDB one-chunk-per-article design decision
- Show: evidence_block_id tracing from screenshot to PDF claim
- Be ready for: "What happens if OCR fails?"

HS Hager DEMO Present: Legal knowledge base + RAG

- Show: ChromaDB with Law 175/2018 articles loaded
- Explain: hard filter by crime_type metadata prevents cross-contamination
- Show: citation validator catching a hallucinated article (demo this live)
- Explain: why multilingual-e5-large over Arabic-only embeddings
- Be ready for: "How did you get the law data?"

ALL DEMO Post-presentation: Next steps

- Prepare 2-minute "What next?" pitch
- TRL 7: 50 real users via ثنرتنإلإا شحابم partnership
- Funding: ITIDA, Flat6Labs, TIEC applications
- Company: GAFI Egypt startup registration
- Revenue: EGP 149/month pro, EGP 2,500/month law firms
- Close: "From a GitHub repo to justice — this is ACEB"

TECHNICAL GUIDES

Quick Reference for Hard Tasks

EasyOCR + Fallback Pattern

OCR

```
import easyocr
reader = easyocr.Reader(["ar","en"], gpu=False)
results = reader.readtext(image_bytes, detail=True)
avg_conf = sum(r[2] for r in results)/len(results) if results else 0
if avg_conf < 0.65:
    # Trigger Google Vision fallback
    from google.cloud import vision
    client = vision.ImageAnnotatorClient()
```

ChromaDB Legal Knowledge Base

RAG

```
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="intfloat/multilingual-e5-large"
)
vectorstore = Chroma.from_texts(
    texts=[a["text"] for a in articles],
```

LangGraph Attacker+Judge Loop

AGENTS

```
from langgraph.graph import StateGraph
from typing import TypedDict, List

class VerState(TypedDict):
    claims: List; evidence_blocks: List
    attacks: List; verdict: str; rounds: int

def attacker_node(state):
```

WeasyPrint Arabic PDF

PDF

```
from weasyprint import HTML
from jinja2 import Environment, FileSystemLoader

env = Environment(loader=FileSystemLoader("templates"))
template = env.get_template("complaint_ar.html")
html = template.render(**case_data, font_path=FONT_PATH)

# CSS required in template:
```

SPRINT COMPLETION TRACKER

Feature Status Checklist

● OCR Service

- ☐ EasyOCR runs on Arabic screenshots
- ☐ Google Vision fallback when confidence <65%
- ☐ Entity extraction: phones, amounts, dates
- ☐ Arabic text normalisation applied

● LLM Classification

- ☐ Crime type: blackmail/scam/threat/defamation
- ☐ Citation-forcing prompt: 0 claims without block_id
- ☐ Confidence score returned
- ☐ Missing evidence list generated

● Multi-Agent Verification

- ☐ Attacker finds unsupported claims
- ☐ Judge: APPROVED / NEEDS_REVISION
- ☐ Auto-remove unsupported claims
- ☐ Max 3 revision rounds enforced

● Arabic PDF

- ☐ WeasyPrint + Amiri font working
- ☐ Arabic RTL text correct
- ☐ All 6 PDF sections rendering
- ☐ Score badge + verification stamp

● Docker + CI/CD

- ☐ docker-compose up starts all 8 services
- ☐ All service healthchecks pass
- ☐ GitHub Actions CI green
- ☐ E2E pytest suite passes

● Legal Knowledge Base

- ☐ Law 175/2018 (45 articles) indexed in ChromaDB
- ☐ Egyptian Penal Code (Art.302-336) indexed
- ☐ Multilingual embeddings loaded
- ☐ Citation validator working

● RAG Retrieval

- ☐ Hard filter by crime_type working
- ☐ Top-5 articles returned per query
- ☐ Citations validated before output
- ☐ Full article text in response

● Evidence Scoring

- ☐ Score 0-100% calculated
- ☐ STRONG/MEDIUM/WEAK grades assigned
- ☐ Breakdown by category returned
- ☐ Missing evidence list accurate

● Legal Chatbot

- ☐ Answers in Arabic with law citations
- ☐ Case-aware (knows uploaded evidence)
- ☐ 0 invented article numbers
- ☐ Session memory working

● Demo + Documentation

- ☐ Cloud URL deployed and accessible
- ☐ Demo video (5 min) recorded
- ☐ README ≥9/10 completeness
- ☐ All 6 KPIs measured and documented

TRL PROGRESS TARGET

